Course Outline

Regression Testing

Week 9, Session 1

Tahun Ajaran 25/26

Institut Teknologi Del
Jl. Sisingamangaraja Sitoluama,
Laguboti 22381 Toba – SUMUT
http://www.del.ac.id

Lecturer : ACB, RCH
Teaching Assistant: -
-- PKPL – 4232201--
-- Sem Genap 2025/2026--

Overview

After studying this material, you will be able to:

- Describe what regression testing is
- Understand the need of regression testing
- Describe issues related to regression testing
- Generate test cases for regression testing of software
- Apply heuristics to reduce the number of test cases required for regression testing
- Students understand about:
  - Test Suite Reduction Methods using
    - Heuristic GRE
    - Harrod Heuristic

Regression Testing: Definition

- A retest of software or its module after it has been modified
- Purpose: to ensure that no adverse effect has been introduced to the software
  - Documents match each other (SRS,SDD, STP, etc)
  - No faults are introduced into the software

Regression Testing (cont'd)

- Different forms of software changes
  - During development
    - Fixing errors found in testing
    - Fixing mistakes in spec, design, etc
    - Adding new requirements as requested by users
  - After delivery
    - Software maintenance
    - Architectural transformation
    - Software re-engineering
- Can software changes be avoided?

Regression Testing: Why is it important?

- Modifying previously validated software may bring in new faults
- Experiences show that the process of repairing previously known faults may be incomplete or bring new errors
- Implication: need to retest the software after it has been modified to ensure that
  - No new faults are introduced
  - The faults to be fixed are actually fixed

Regression Testing: Why is it important?

- As long as the software needs to be modified, regression testing needs to be performed
- The process of programming testing debugging continues as many times as the software behaves according to the specifications
- The test suite designed for verifying the software behaving according to its specifications may need to be executed as many times as it needs

Regression Testing: Why is it important?

- It is performed quite often in practice during and after the software development
- It is important for companies which
  - Develop several similar products with or without reusing parts of its products
  - Maintain large programs over extended periods of time
  - Develop software which is under constant evolution

Regression Testing

- Costs incurred in regression testing include time spent by the computer (when its cost is not negligible) :
  - On generating test cases for the newly modified parts
  - On revalidating the previous test suite
  - On executing the test suite
  - On checking the results on test execution
  - On tracking failures to the module or requirements

Issues in Regression Testing

- Related to testing approach
  - Retest all
  - Retest selectively
- Related to test case
  - Revalidating existing test cases
    - Identifying obsolete test cases
    - Reusing valid test cases
  - Generating new test cases for the modified software
- Related to test suite
  - Test suite reduction

Retest All

- Retest all testing requirements of the software
- It is the most safe option
- It may be very time consuming
- It is the recommended approach unless the cost is too expensive
- Need to select test cases to retest all parts of the software
- Need to generate new test cases to test the newly modified parts (including the newly introduced parts) of the software

Retest All-Example

Changing the specification of a method CalculateSalary() in a financial system causes the retest to the whole system

Retest selectively-Example

Changing the specification of a method CalculateSalary() in a financial system causes the retest all methods related to CalculateSalary()

Revalidating Existing Test Cases

- Test cases need to be re-validated because:
  - Specifications may be changed
  - Design may be changed
  - Program source may be changed
- It is done by checking whether the input and the expected output match accordingly
- It is mainly a manual process, time consuming, and expensive
- Valid test cases (where their inputs and expected outputs are matched) can be used for regression testing
- Invalid test cases may be discarded or maybe used to check whether a program can handle the invalid situations

Identifying obsolete test cases

- An *obsolete* test case is a test case that is not needed
- Test cases are obsolete because
  - Their corresponding requirements are no longer valid
- Obsolete test cases need to be identified and “discarded”
- A better approach is to store these obsolete test cases for later uses (they may be valuable for the later version)

Generating New Test cases

- New test cases need to be generated because:
  - ❑ new specifications may be introduced
  - ❑ new design may be introduced
  - ❑ new code may be introduced
- Test case generation is done as if it is for the initial testing
- Test case generation and preparation are very time consuming and expensive

Test Suite Reduction

- A test suite is a set of test cases that satisfies all the desired testing requirements
- A representative set is a subset of the test suite that satisfies all the desired testing requirements as the test suite
- Given a test suite T that satisfies a set of testing requirements R, the test suite reduction problem seeks to find a representative set of T (in satisfying the same set of testing requirements R)
- The problem of obtaining an optimal (in terms of smallest size) representative set of a test suite is NP complete
- Several heuristics exist for finding an approximate solution to the optimization problem

Test Suite Reduction

- When the software has been modified, the test suite may need to be modified as well
- Managing the test suite involves:
  - Identifying obsolete test cases
  - Revalidating existing test cases for new requirements
  - Designing new test cases for testing the changes
  - Eliminating unnecessary test cases to reduce the overhead of re-executing and maintaining the test suites
- The test suite may grow significantly if effort is not taken to remove unnecessary test cases

Unnecessary Test Cases

- Changing the specifications of a program may cause some testing requirements to vanish and may create some new testing requirements
- Adding new test cases to satisfy new testing requirements may render an existing test case redundant if other test cases in the expanded test set together already satisfy all the same test requirements

TSR Heuristic - Notation

- T = the test suite under consideration
- R = the set of testing requirements to be satisfied
- t_i = an arbitrary test case in T
- r_j = an arbitrary testing requirement in R
- R_k = the set of testing requirement satisfied by exactly *k* test cases

TSR Heuristic – Notation (cont'd)

- Test(r_j) = the set of test cases satisfying the requirement r_j
- Test(R_k) = the set of test cases satisfying the requirements in R_k
- The set R_1 contains all testing req which are satisfied by only one test case. Such a test case is called *essential test case*

Heuristic GRE

- There are 3 strategies:
  1. G: greedy

     G: pick test cases satisfying the largest number yet-to-be satisfied testing requirements and mark the satisfied reqs
  2. R: the 1-to-1 redundancy strategy

     R: removes all 1-to-1 redundancy strategy. t is called 1-to-1 redundancy if there is another test case t' where t <> t' such that t' satisfies all reqs satisfied by t
  3. E: the essential strategy

     E: select all essential test cases and mark the satisfied reqs. t is an essential test case if there is a testing req that can be satisfied **only** by t.
- **It applies E and R alternatively until neither of them can be applied. Under this circumstances, G is applied.**

Heuristic GRE - Example

- Test Suite, T= {t1,…,t7}
- Testing Requirement, R={r1,…,r8}
- Test case satisfying the requirements is given by the satisfiability relation in next slide

Satisfiability relation

	r_1	r_2	r_3	r_4	r_5	r_6	r_7	r_8
t_1	X		X		X	X		
t_2			X					X
t_3			X	X			X	X
t_4					X		X	X
t_5	X	X						
t_6				X		X		
t_7							X	X

Heuristic GRE-example

- Apply E first. Select t_5
- Mark r_1 and r_2

Satisfiability relation

	r_1	r_2	r_3	r_4	r_5	r_6	r_7	r_8
t_1	X		X		X	X		
t_2			X					X
t_3			X	X			X	X
t_4					X		X	X
t_5	X	X						
t_6				X		X		
t_7							X	X

Satisfiability relation

			r_3	r_4	r_5	r_6	r_7	r_8
t_1			X		X	X		
t_2			X					X
t_3			X	X			X	X
t_4					X		X	X
t_6				X		X		
t_7							X	X

Heuristic GRE-example

- R is applied
- Remove t_2 … why ?
- Remove t_7 … why ?

Satisfiability relation

			r_3	r_4	r_5	r_6	r_7	r_8
t_1			X		X	X		
t_2			X					X
t_3			X	X			X	X
t_4					X		X	X
t_5								
t_6				X		X		
t_7							X	X

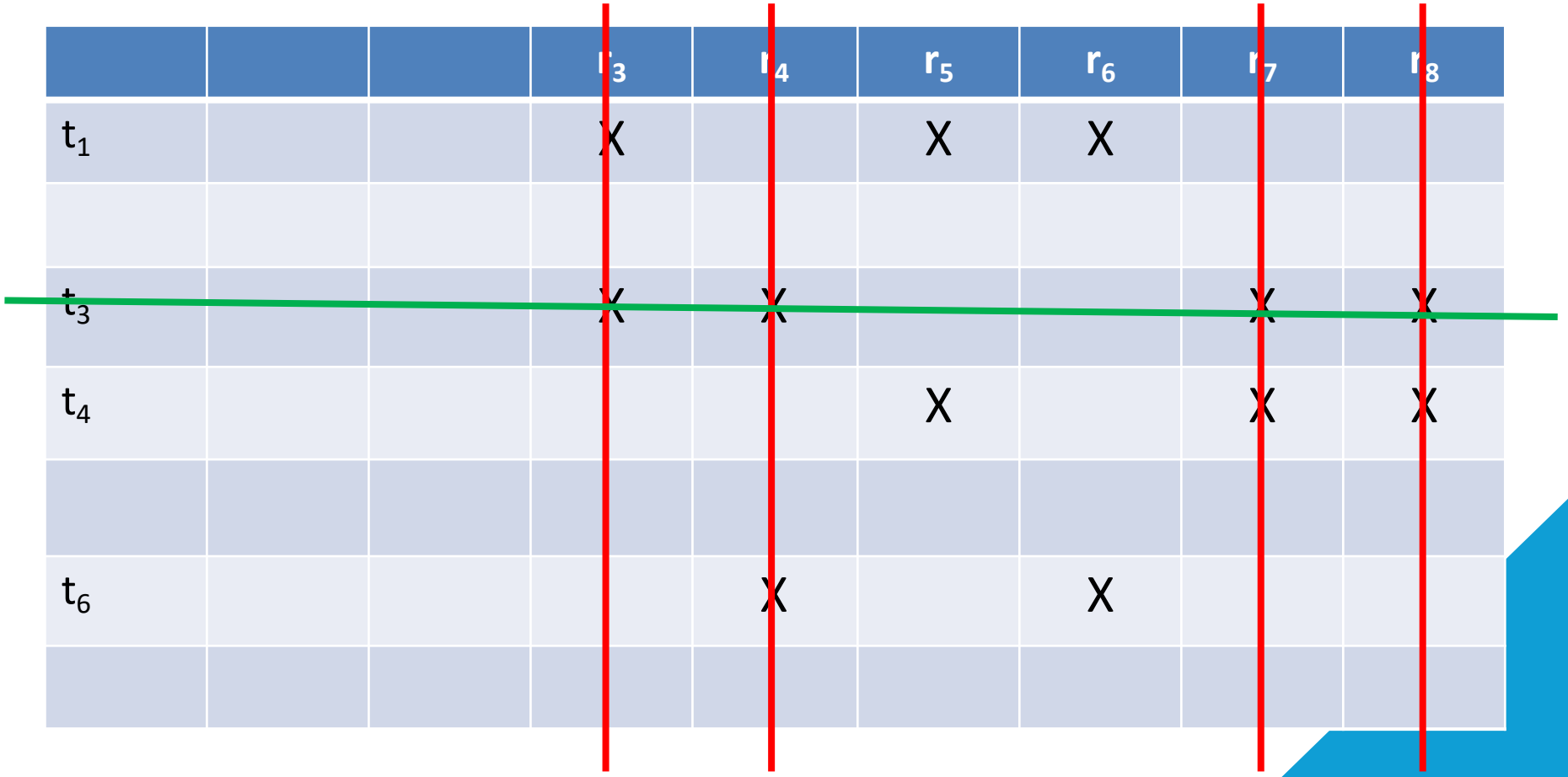
Satisfiability relation

			r_3	r_4	r_5	r_6	r_7	r_8
t_1			X		X	X		
t_3			X	X			X	X
t_4					X		X	X
t_6				X		X		

Heuristic GRE-example

- Both E and R cannot be applied
- Apply G
- Select t_3 (most cover r)
- Mark r_3, r_4, r_7 and r_8

Satisfiability relation



			r_3	r_4	r_5	r_6	r_7	r_8
t_1			X		X	X		
t_3			X	X			X	X
t_4					X		X	X
t_6				X		X		

Satisfiability relation

					r_5	r_6		
t_1					X	X		
t_4					X			
t_6						X		

Heuristic GRE-example

- R is applied
- Remove t_4 … why ?
- Remove t_6 … why ?

Satisfiability relation

					r_5	r_6		
t_1					X	X		
t_4					X			
t_6						X		

Heuristic GRE-example

- Apply E. Select t_1
- Mark r_5 and r_6

Satisfiability relation

					r_5	r_6		
t_1^+					X	X		

Steps Summary

- E : select t_5
- R : remove t_2 and t_7
- G : select t_3
- R : remove t_4 and t_6
- E : select t_1

Solution

- The representative set is : t_5, t_3, t_1
- NOTES
  - E: t_5
  - G: t_3
  - E: t_1

Test Suite: Result

Before

	r_1	r_2	r_3	r_4	r_5	r_6	r_7	r_8
t_1	X		X		X	X		
t_2			X					X
t_3			X	X			X	X
t_4					X		X	X
t_5	X	X						
t_6				X		X		
t_7							X	X

After

	r_1	r_2	r_3	r_4	r_5	r_6	r_7	r_8
t_1	X		X		X	X		
t_3			X	X			X	X
t_5	X	X						

Harrold's Heuristic, Heuristic H

- Intuition:
  - Generally, if i < j, test cases in Test(R_i) are "more essential" than those in Test(R_j)
  - if i < j, test cases in Test(R_i) are picked before test cases in Test(R_j)
- Pick all test cases in Test(R_1) then mark all testing reqs in R satisfied by these test cases
- For each i = 2, 3, …, d
  - pick the test case t in Test(R_i) satisfying the largest number of yet-to-be satisfied testing reqs in R_i. In case of a tie situation, pick the one that satisfies largest number of yet-to-be satisfied testing reqs in R_{i+1}, In case of a tie situation again, pick the one that satisfies the largest testing reqs in R_{i+2}, and if is still tie until checking reqs in R_d so please an arbritary/random test case. (d is the maximum number of test cases in T that can satisfy a requirement in R)
  - Mark the testing reqs in R satisfied by t
  - Repeat until all requirements in R_i satisfied

Heuristic H - example

- Test suite T = {t_1, …, t_7}
- Testing Requirement, R = {r_1, …, r_8}
- Test case satisfying the requirements is given by the satisfiability relation in next slide

Satisfiability relation

	r_1	r_2	r_3	r_4	r_5	r_6	r_7	r_8
t_1	X		X		X	X		
t_2			X					X
t_3			X	X			X	X
t_4					X		X	X
t_5	X	X						
t_6				X		X		
t_7							X	X

Heuristic H-example

- $R_1 = \{r_2\}$
- $R_2 = \{r_1, r_4, r_5, r_6\}$
- $R_3 = \{r_3, r_7\}$
- $R_4 = \{r_8\}$

r_i	Test (r_i)
r_1	$\{t_1, t_5\}$
r_2	$\{t_5\}$
r_3	$\{t_1, t_2, t_3\}$
r_4	$\{t_3, t_6\}$
r_5	$\{t_1, t_4\}$
r_6	$\{t_1, t_6\}$
r_7	$\{t_3, t_4, t_7\}$
r_8	$\{t_2, t_3, t_4, t_7\}$

Satisfiability relation

- $R_{1 =} \{r_2\}$ and Test $(R_1) = \{t_5\}$. Then select t_5, mark r_1 and r_2
- $R_{2=} \{r_1, r_4, r_5, r_6\}$ where r_1 is already satisfied. Test $(R_2) = t_1$ can satisfy 2 unsatisfied reqs in R_2, t_3 can satisfy 1 unsatisfied reqs, t_4 can satisfy 1 unsatisfied reqs, t_5 has been selected, t_6 can satisfy 2 unsatisfied reqs. Then, we check reqs in R_3 that can be satisfied by t_1 and t_6. We choose t_1 as t_1 can satisfy 1 and t_6 none. Then we mark r_3, r_5 and r_6
- Now we still have r_4 in R_2 that has not been marked yet and Test$(r_4) = \{t_3, t_6\}$. Both of them can only satisfy one req in R_2.. Then, we check reqs in R_3 that can be satisfied by t_3 and t_6. We choose t_3 as t_3 can satisfy 1 and t_6 none. Then we mark r_4, r_7 and r_8
- Now all req are satisfied
- DONE. A representative set is $\{t_5, t_1, t_3\}$

Thank you